

INGENIERÍA DE SOFTWARE 1 · UNIDAD 8

Tarea previa: prepara tu equipo y tu computadora

Requisito para entrar al laboratorio de pruebas y versionamiento

Campo	Detalle
Unidad	U8 · Pruebas de Software (con puente a U9: administración de la configuración)
Tipo de material	Tarea previa obligatoria. Habilita el laboratorio calificado.
Nivel	Básico
Tiempo estimado	45 a 60 minutos (individual; el paso 3 lo hace una sola persona por equipo)
Entrega	Antes del laboratorio: URL del repositorio + salida de <code>check_setup.py</code> + tu commit (ver paso 7)

Por qué es obligatoria

El laboratorio dura **60 minutos** y está diseñado para practicar pruebas y versionamiento, no para instalar programas.

Si llegas sin todo listo, tu equipo pierde minutos que el laboratorio no tiene. Haz esta tarea con calma y con tiempo.

Antes de empezar: qué vas a lograr

- Tener **Python**, **pytest** y **Git** funcionando en tu laptop.
- Tener tu **cuenta de GitHub** lista y autenticada.
- Crear (o unirse a) el **repositorio de tu equipo** desde la plantilla del curso.
- Ejecutar tu **primera prueba en rojo**, arreglarla y subir tu primer commit.

Paso 1 · Instala y configura (todos, 10 min)

1.1 Programas necesarios

Programa	Versión	Dónde obtenerlo	Nota
Python	3.10 o superior	python.org/downloads	En Windows marca Add python.exe to PATH durante la instalación.
Git	cualquier versión reciente	git-scm.com/downloads	Deja las opciones por defecto.

Programa	Versión	Dónde obtenerlo	Nota
Editor de código	VS Code (recomendado)	code.visualstudio.com	Puedes usar otro editor si ya tienes uno.

1.2 Verifica que quedaron instalados

Abre una terminal nueva (PowerShell en Windows, Terminal en macOS o Linux) y ejecuta:

```
python --version
git --version

# macOS/Linux: si "python" no existe, usa python3 --version
# Windows: si "python" no existe, prueba py --version
```

En todo este documento escribimos `python`. Si en tu equipo funciona `python3` o `py`, úsalo en su lugar **en todos los comandos**.

1.3 Configura tu identidad en Git (una sola vez)

```
git config --global user.name "Tu Nombre Apellido"
git config --global user.email "correo-de-tu-cuenta-github@ejemplo.com"
git config --global pull.rebase true
```

La tercera línea evita que Git abra editores de texto inesperados cuando trabajes con tu compañero. Es importante.

Paso 2 · Cuenta de GitHub y autenticación (todos, 5 min)

1. Crea tu cuenta en **github.com** si aún no tienes una. Anota tu **usuario**.
2. Configura cómo se identificará Git ante GitHub. Elige **una** opción:

Opción	Cómo	Cuándo
A · GitHub CLI (recomendada)	Instala desde cli.github.com y ejecuta <code>gh auth login</code> . Responde: GitHub.com → HTTPS → Y (autenticar Git con tus credenciales) → Login with a web browser.	Funciona igual en Windows, macOS y Linux.
B · Git Credential Manager	En Windows viene incluido con Git. La primera vez que hagas <code>git push</code> se abrirá el navegador para iniciar sesión.	Solo Windows.

Ojo con la contraseña

La contraseña de tu cuenta de GitHub **ya no sirve** para `git push` por HTTPS (GitHub la eliminó en agosto de 2021). Si Git te la pide y falla, usa la opción A.

Paso 3 · Crea el repositorio del equipo (una persona por equipo, 5 min)

Tu docente te asignó un número de equipo (**gNN**, por ejemplo g07) y compartió el enlace de la plantilla. Decidan quién será el **dueño del repositorio**; solo esa persona hace este paso.

1. Abre el enlace de la plantilla `pagofacil-lab-plantilla` y pulsa el botón verde **Use this template** → **Create a new repository**.
 - a. <https://github.com/pedagawo/pagofacil-lab-plantilla>
1. En **Owner** deja tu usuario. En **Repository name** escribe exactamente `pagofacil-lab-gNN` (con tu número de equipo).
2. Elige visibilidad **Public** (el docente necesita poder revisarlo) y pulsa **Create repository**.
3. Ve a **Settings** → **Collaborators** → **Add people** e invita a tu(s) compañero(s) por su usuario de GitHub.
4. Tus compañeros deben **aceptar la invitación** (llega por correo o en sus notificaciones de GitHub).

Verifica

La URL de tu repositorio debe verse así: `https://github.com/<usuario-dueño>/pagofacil-lab-g07`.
Cópiala: la necesitarás para la entrega.

Paso 4 · Clona y verifica tu entorno (todos, 10 min)

Todos los integrantes (incluido el dueño) clonan el repositorio en su laptop:

```
git clone https://github.com/<usuario-dueño>/pagofacil-lab-gNN.git
cd pagofacil-lab-gNN
python -m pip install -r requirements.txt
python check_setup.py
```

La salida debe terminar con **RESULTADO: LISTO PARA EL LABORATORIO**. Si aparece alguna línea `[FALLA]`, el propio verificador te dice cómo arreglarla. Corrígela y vuelve a ejecutarlo.

```
[ OK ] Python          3.12.3
[ OK ] pytest          9.1.1
[ OK ] git              git version 2.43.0
[ OK ] git identidad    Ana Pérez <ana@ejemplo.com>
[ OK ] git pull.rebase  true
[ OK ] Repositorio de equipo pagofacil-lab-g07
[ OK ] Conexión con GitHub se puede leer el remoto
[ OK ] Rama actual      main
RESULTADO: LISTO PARA EL LABORATORIO
```

Paso 5 · Tu primera prueba en rojo (todos, 15 min)

Esta mini-práctica te enseña **lo único que necesitas saber de pytest** para el laboratorio: correr pruebas, leer un fallo, corregir y agregar una prueba.

5.1 Corre las pruebas de calentamiento

```
python -m pytest tarea_previa -v
```

Verás **1 failed, 2 passed**. Que una prueba falle aquí es lo esperado. Lee el reporte:

Parte del reporte	Qué significa
FAILED ...::test_justo_18_es_mayor_de_edad	Nombre de la prueba que falló. Los nombres cuentan qué se esperaba.
> assert es_mayor_de_edad(18) is True	La línea que falló (marcada con >).
E assert False is True	Lo que devolvió el código (<code>False</code>) frente a lo esperado (<code>True</code>).
1 failed, 2 passed	Resumen final. Tu objetivo es que diga solo <code>passed</code> .

5.2 Corrige el defecto

Abre `tarea_previa/edad.py`. La regla dice que **18 años cumplidos ya es mayor de edad**. Compara esa frase con el operador de la comparación y corrígelo. Vuelve a correr `python -m pytest tarea_previa -v` hasta ver todo en verde.

5.3 Agrega una prueba parametrizada

Una **prueba parametrizada** ejecuta la misma verificación con varios datos. Agrega esto al final de `tarea_previa/test_edad.py`:

```
import pytest

@pytest.mark.parametrize("edad, esperado", [(17, False), (18, True), (19, True)])
def test_frontera_de_edad(edad, esperado):
    assert es_mayor_de_edad(edad) is esperado
```

Vuelve a correr las pruebas: deben ser **6 passed**. Fíjate en los valores `17`, `18` y `19`: se llaman **valores límite** y son donde más se esconden los defectos. Los usarás mucho en el laboratorio.

Paso 6 · Tu primer commit y tu primer push (todos, 10 min)

Trabajen **de uno en uno** (avísense por chat) para evitar choques. Cada integrante hace lo siguiente en su laptop:

1. **Actualiza** antes de tocar nada: `git pull`
2. Haz tu corrección de la mini-práctica (solo el primer integrante que la termine hace este commit; los demás pueden repasarla sin subirla):

```
git status
git add .
git commit -m "fix(tarea-previa): incluir 18 años como mayor de edad"
git push
```

- Ahora **cada integrante** registra su presencia: con tu editor crea el archivo `equipo/<tu-usuario-github>.txt` (crea la carpeta `equipo` si no existe) que contenga tu nombre y tu carnet. **No lo crees desde la terminal con el comando echo**: en Windows guardaría caracteres extraños.
- Sube tu archivo:

```
git pull
git add .
git commit -m "chore(equipo): agregar integrante <tu-usuario-github>"
git push
```

Si git push dice "rejected"

Significa que tu compañero subió algo antes que tú. No pasa nada: ejecuta `git pull` y luego `git push` otra vez. Con la configuración del paso 1.3 no se abrirá ningún editor.

Formato de los mensajes de commit

Usamos **Conventional Commits**: `tipo(alcance): descripción corta en presente`.

Tipos que usaremos: `fix` (corrige un defecto), `feat` (funcionalidad nueva), `docs` (documentación), `test` (pruebas), `chore` (tareas menores).

Bien: `fix(comision): incluir Q100 exacto en el tramo sin comisión`. Mal: `arreglé cambios, fix`.

Referencia rápida

Git: los 8 comandos que necesitas

Quiero...	Comando
Ver en qué estado está mi trabajo	<code>git status</code>
Traer lo que subió mi compañero	<code>git pull</code>
Preparar todos mis cambios para guardarlos	<code>git add .</code>
Guardar una versión de mi trabajo (commit)	<code>git commit -m "fix(alcance): descripción"</code>
Subir mis commits a GitHub	<code>git push</code>
Ver el historial resumido	<code>git log --oneline --decorate</code>
Marcar una versión (etiqueta)	<code>git tag -a v1.0.0 -m "descripción"</code>
Subir las etiquetas a GitHub	<code>git push --tags</code>

pytest en 60 segundos

Necesito...	Ejemplo
Correr todas las pruebas	<code>python -m pytest</code>

Necesito...	Ejemplo
Ver el detalle de cada prueba	<code>python -m pytest -v</code>
Correr solo una prueba (por parte del nombre)	<code>python -m pytest -k monto_exacto -v</code>
Verificar un valor	<code>assert calcular(5) == 10</code>
Verificar que se lance un error	<code>with pytest.raises(ValueError) :</code> y dentro la llamada
Probar muchos datos con una sola prueba	<code>@pytest.mark.parametrize("entrada, esperado", [(1, 2), (3, 4)])</code>

Problemas frecuentes

Síntoma	Causa probable	Solución
<code>python</code> no se reconoce	No está en el PATH o se llama distinto	Prueba <code>py</code> (Windows) o <code>python3</code> (macOS/Linux). Si sigue igual, reinstala Python marcando *Add to PATH*.
No module named <code>pytest</code>	Falta instalarlo	<code>python -m pip install -r requirements.txt</code>
<code>pytest</code> no se reconoce	El ejecutable no está en el PATH	Usa siempre <code>python -m pytest</code> .
No module named <code>pagofacil</code>	Ejecutas desde otra carpeta	Entra a la carpeta raíz del repositorio (donde está <code>pytest.ini</code>).
<code>Author identity unknown</code>	Falta configurar Git	Repite el paso 1.3.
<code>Authentication failed</code> o piden contraseña	GitHub no acepta contraseña por HTTPS	Usa <code>gh auth login</code> (paso 2, opción A).
<code>git push rejected</code>	Tu compañero subió antes	<code>git pull</code> y luego <code>git push</code> .
Se abrió una pantalla extraña (vim)	Git pidió un mensaje sin <code>-m</code>	Escribe <code>:q!</code> y pulsa Enter; repite el commit con <code>-m "mensaje"</code> .
<code>check_setup.py</code> dice que estás en la PLANTILLA	Clonaste la plantilla, no el repo de tu equipo	Clona la URL que termina en <code>pagofacil-lab-gNN</code> .